CURSO ANDROID · SESIÓN 10

Android avanzado y seguridad

Material de estudio

Plataforma Android, trabajo persistente y protección de datos explicados con ejemplos de la app PokeAPI.

Qué encontrarás

Conceptos desarrollados, comparaciones, fragmentos de Kotlin y Compose, recorrido de laboratorio, checklist de seguridad, ejercicios y respuestas comentadas.

Conocimientos previos

Kotlin, Jetpack Compose, Retrofit, Clean Architecture, Koin y uso básico de persistencia local.

Los ejemplos breves indican su contexto y omiten imports cuando se enumeran debajo. La integración completa de WorkManager se encuentra en el proyecto del curso. Las medidas de seguridad deben elegirse según el riesgo del dato y la operación.

Contenido y uso del material

Lee primero los conceptos de plataforma. Después sigue el recorrido de WorkManager y revisa seguridad. Resuelve la autoevaluación antes de consultar sus respuestas. Los números entre corchetes enlazan a las fuentes oficiales.

Lifecycle y ejecución de una app	3
Process death y estado restaurable	4
Compose, coroutines y flujo de estado	5
Permisos y mínimo privilegio	6
Intents y componentes exportados	7
Deep links y Android App Links	8
Elección del trabajo en segundo plano	9
WorkManager: estados, condiciones y retry	10
Implementación de un CoroutineWorker	11
Solicitudes únicas e idempotencia	12
Integración con Clean Architecture y Koin	13
Estado observable y demo de PokeAPI	14
Pruebas y diagnóstico del trabajo	15
Almacenamiento y clasificación de datos	16
Android Keystore y AES-GCM	17
BiometricPrompt y autenticación local	18
HTTPS, TLS y certificate pinning	19
Tokens, sesiones y autorización	20
R8, logs y secretos	21
Screenshots y clipboard	22
OWASP MASVS y revisión por riesgos	23
Checklist y práctica de consolidación	24
Autoevaluación	25
Respuestas comentadas	26
Glosario	27
Fuentes oficiales: plataforma y trabajo	28
Fuentes oficiales: datos y seguridad	29

Lifecycle y ejecución de una app

Una aplicación Android no controla por completo cuánto tiempo permanece en memoria. El usuario puede cambiar de pantalla, abrir otra app o volver después de un periodo de inactividad. Android administra los recursos y puede recrear componentes o terminar el proceso.

Una **Activity** es un componente que aloja una experiencia de interfaz. En una app con Compose, una misma Activity puede contener varios destinos de navegación. La Activity, el proceso y cada composable tienen duraciones diferentes: no deben tratarse como si fueran el mismo objeto. [1, 3]

Callbacks y responsabilidades

Callback	Situación habitual	Responsabilidad orientativa
onCreate	Se crea una instancia	Preparar UI y restaurar estado disponible
onStart	Empieza a ser visible	Activar observación ligada a visibilidad
onResume	Está en interacción principal	Habilitar recursos que exigen ese estado
onPause	Pierde el estado principal	Ajustar recursos vinculados a interacción
onStop	Ya no es visible	Detener recursos que requieren visibilidad
onDestroy	Se destruye la instancia	Liberar recursos de esa instancia

La secuencia cambia según la acción del usuario. Una Activity pausada puede seguir visible, por ejemplo en multiventana. **onDestroy no está garantizado cuando Android mata el proceso.** Por ello, una operación importante debe persistirse antes de depender del cierre. [1]

Ejemplo aplicado

Si el usuario marca un Pokémon como favorito, guardar la decisión al producirse la acción. Al regresar a la pantalla, consultar la persistencia. Mantener la selección solo en una variable de la Activity hace que pueda perderse al recrearla.

Para razonar: cerrar una pantalla, rotar el dispositivo y terminar el proceso pueden exigir distintas respuestas. Antes de elegir una API, identificar qué objeto desaparece y qué información se necesita recuperar.

Process death y estado restaurable

Process death significa que termina el proceso que ejecuta la app. Los objetos en memoria desaparecen, incluidos ViewModels y singletons del contenedor de dependencias. Si el usuario vuelve, se crean nuevas instancias. Un singleton de Koin tiene alcance dentro del proceso, no persistencia entre procesos.

Hay que distinguir el estado de presentación, los datos de negocio y las operaciones pendientes. Un texto de búsqueda puede restaurarse con estado guardado; una lista descargada pertenece al almacenamiento; una sincronización durable necesita un planificador. [2, 5]

Mecanismo	Conserva principalmente	Límite importante
remember	Valores entre recomposiciones	Se pierde al salir de esa composición
ViewModel	Estado durante recreación por configuración	Muere con el proceso
rememberSaveable	Estado pequeño de UI restaurable	Depende del estado guardado y de la tarea
SavedStateHandle	Estado pequeño asociado al ViewModel	No sustituye persistencia durable
Room o archivos	Datos que deben persistir	Requieren diseño de lectura y escritura

Reconstrucción de la experiencia

Guardar identificadores y decisiones pequeñas: consulta de búsqueda, pestaña o ID seleccionado. Reconstruir listas y detalles desde el repositorio. Evitar grandes colecciones, imágenes o secretos en el mecanismo de estado guardado. [2]

```
// Dentro de un composable. Imports de Compose omitidos.
var query by rememberSaveable { mutableStateOf("") }

// Dentro de un ViewModel que recibe SavedStateHandle.
val query = savedStateHandle.getStateFlow("query", "")
fun updateQuery(value: String) {
    savedStateHandle["query"] = value
}
```

Los fragmentos ilustran alternativas de propiedad del estado. No es necesario mantener dos fuentes independientes para la misma consulta. Una restauración puede no ocurrir si se elimina la tarea o se borran datos.

Error frecuente: usar rotación como única prueba de process death. La rotación suele recrear la Activity conservando el ViewModel. Para el caso de proceso, se debe probar una terminación con la app en background y reconstruir después.

Compose, coroutines y flujo de estado

La **recomposición** actualiza la interfaz ejecutando nuevamente los composables que lo requieren. Puede ocurrir repetidamente. El cuerpo de un composable debe describir la UI a partir del estado, sin lanzar solicitudes de red como efecto incidental de cada ejecución. [3]

Una coroutine permite suspender operaciones sin bloquear el hilo que necesita atender otras tareas. Sin embargo, su duración depende del alcance: una operación en viewModelScope se cancela cuando se elimina el ViewModel. Usar coroutines no convierte automáticamente el trabajo en persistente.

Responsabilidades en la app PokeAPI

- **UI:** muestra búsqueda, lista, errores y acciones disponibles.
- **ViewModel:** transforma flujos en estado y recibe acciones de la pantalla.
- **Caso de uso:** expresa una operación del dominio, como actualizar la lista.
- **Repositorio:** coordina acceso remoto y local.
- **Room:** conserva el catálogo y emite sus cambios.

```
@Composable
fun PokemonScreen(vm: PokemonListViewModel) {
    val ui by vm.state.collectAsStateWithLifecycle()
    // La UI usa ui y envía eventos al ViewModel.
    Text("Guardados: ${ui.pokemon.size}")
}
```

El ejemplo utiliza el ViewModel del proyecto, Text de Material y la extensión de lifecycle-runtime-compose. collectAsStateWithLifecycle adapta la recolección al lifecycle. El repositorio puede seguir siendo la fuente de datos aunque cambie el estado de la pantalla. [3]

Un modelo útil para estudiar

El botón emite un evento. El ViewModel solicita una acción. El repositorio actualiza la persistencia y el flujo emite datos nuevos. La pantalla refleja ese estado. Así se evita mantener una lista distinta en el Worker y otra en la UI.

Ejercicio breve: una búsqueda se ejecuta tres veces al cambiar un texto de color en la UI. Revisar si la llamada se colocó en el cuerpo del composable. La solución depende de la intención: una acción explícita o un efecto con claves adecuadas, manteniendo la coordinación en la capa correspondiente.

Permisos y mínimo privilegio

Un permiso controla acceso a capacidades o datos protegidos de Android. Los permisos normales y los peligrosos tienen procesos distintos de concesión. **INTERNET se declara en el manifest y no requiere diálogo runtime.** Solicitar datos a PokeAPI no justifica cámara, ubicación o acceso amplio al almacenamiento. [4]

```
<uses-permission android:name="android.permission.INTERNET" />
```

El principio de **mínimo privilegio** consiste en solicitar solo el acceso necesario para una función. Antes de agregar un permiso, buscar una API del sistema que resuelva la tarea con menor acceso, como Photo Picker para seleccionar imágenes.

Solicitud contextual y rechazo

La app debe explicar el propósito en el momento de uso y manejar permiso concedido, rechazado o revocado. No asumir que una autorización permanece para siempre. Si la función es opcional, conservar el resto de la experiencia. [4]

```
// Fragmento dentro de un composable. CAMERA debe estar en manifest.
val cameraPermission = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestPermission()
) { granted ->
    // Actualizar estado y continuar solo si existe autorización.
}
Button(onClick = {
    cameraPermission.launch(Manifest.permission.CAMERA)
}) { Text("Usar cámara") }
```

Este fragmento muestra el contrato de solicitud, no una pantalla completa de cámara. Una implementación debe comprobar la autorización actual, explicar el contexto y ofrecer recuperación cuando corresponda.

Decisiones sobre PokeAPI

Función	Acceso esperado
Descargar nombres y detalles	INTERNET
Guardar catálogo con Room interno	No necesita almacenamiento externo
Seleccionar una imagen del usuario	Evaluar Photo Picker
Capturar una foto dentro de la app	Revisar permiso y API de cámara

Para comprobar comprensión: si una función no necesita ubicación, pedirla como requisito de entrada aumenta acceso sin aportar valor. La revisión del manifest debe acompañar la revisión del código que usa cada permiso.

Intents y componentes exportados

Un **Intent** comunica una acción entre componentes. El Intent explícito nombra el destino. El implícito describe una acción que Android intenta resolver mediante componentes compatibles. Un caso habitual es compartir texto con otra app. [6]

```
val send = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Pokémon: pikachu")
}
startActivity(Intent.createChooser(send, "Compartir Pokémon"))
```

El ejemplo usa android.content.Intent desde una Activity. El tipo MIME indica cómo interpretar el contenido. Los extras transportan datos, pero no demuestran identidad ni autorización.

Superficie de entrada

Una Activity exportada puede recibir llamadas externas. Un intent-filter publica coincidencias de acciones, categorías y datos, pero **no constituye por sí solo un control de acceso**. Revisar android:exported y exponer únicamente los componentes necesarios. [6]

Si una pantalla recibe un ID de recurso privado, debe comprobar la sesión y solicitar al servidor la autorización correspondiente. Un extra como isAdmin=true nunca debe decidir privilegios.

Reglas para manejar entradas

- Validar tipos, formatos, tamaños y valores aceptados.
- Evitar reenviar Intents anidados sin comprobarlos.
- Compartir archivos mediante mecanismos apropiados, como FileProvider.
- Tratar parámetros externos como entrada no confiable aunque parezcan internos.

Caso para analizar: una Activity de detalle abre directamente cualquier ID recibido. Para Pokémon públicos, el riesgo puede ser limitado a errores o rutas inesperadas. Si el mismo patrón se usa para documentos privados, el backend debe impedir que cambiar el ID dé acceso a documentos de otra persona.

La seguridad se diseña alrededor del recurso y de sus permisos. Una navegación exitosa no significa una operación autorizada.

Deep links y Android App Links

Un **deep link** abre un destino específico de una aplicación. Puede facilitar compartir contenido, llegar desde una notificación o continuar un flujo. Un esquema propio como pokecourse://pokemon/25 puede registrarse también en otra app. Un **Android App Link** utiliza HTTPS y una asociación verificada con un dominio. [7]

Enlace	Característica	Decisión de seguridad
Esquema propio	No demuestra propiedad exclusiva	Validar y prever posibles competidores
Web HTTPS	Puede abrir web o app según resolución	Comprobar ruta y parámetros
App Link verificado	Asocia dominio y aplicación	Mantener validación y autorización

La asociación requiere el archivo assetlinks.json servido por el dominio y los datos correctos de la app y su certificado de firma. android:autoVerify solicita verificación, pero no garantiza que haya funcionado. Probar el estado real y considerar la firma de distribución. [8]

Dominio propio y datos externos

La app del curso consume PokeAPI, pero no controla el dominio de ese proveedor. Para enlaces propios se necesita un dominio bajo control del proyecto. curso.example.com se usa aquí solo como ejemplo.

```
// Import android.net.Uri. El dominio es ilustrativo.
fun pokemonId(uri: Uri): Int? {
    if (uri.scheme != "https") return null
    if (uri.host != "curso.example.com") return null
    val parts = uri.pathSegments
    if (parts.size != 2 || parts[0] != "pokemon") return null
    return parts[1].toIntOrNull()?.takeIf { it > 0 }
}
```

El validador acepta una estructura definida antes de navegar. Debe invocarse tanto al arranque como al recibir un Intent nuevo si se reutiliza la Activity. Probar ID inválido, ruta extra y dominio ajeno.

Límite: validar una URL no autoriza acceso a recursos privados. Evitar tokens de sesión en enlaces, porque pueden terminar en historiales, logs o servicios de análisis.

Elección del trabajo en segundo plano

El primer criterio es qué debe ocurrir si la pantalla desaparece. Una solicitud para una UI puede cancelarse al abandonarla. Una actualización de caché que debe completarse después puede necesitar trabajo persistente. **WorkManager administra trabajo que puede diferirse y conserva su programación**, sujeto al sistema. [9]

Necesidad	Herramienta para evaluar	Límite
Consulta ligada a una pantalla	Coroutine en alcance adecuado	El alcance puede cancelarse
Sincronización durable de caché	WorkManager	Inicio inexacto y restricciones
Trabajo continuo visible al usuario	Servicio en primer plano	Notificación y restricciones de plataforma
Evento que necesita hora precisa	Alarmas según caso de uso	Permisos y requisitos específicos

Persistente no significa inmediato

WorkManager puede recuperar programación tras terminación ordinaria del proceso o reinicio. El dispositivo puede retrasar trabajo por batería, cuotas, condiciones y políticas de Android. Una app detenida mediante force-stop no continúa normalmente hasta su reactivación. [9]

En la app del curso, descargar otra vez el catálogo puede esperar. Por eso el ejemplo usa WorkManager. Una animación o la respuesta inmediata de un botón no debería depender de su planificador.

Diferencias que evitan errores

Hilo: recurso de ejecución. **Coroutine:** unidad de concurrencia con suspensión. **Alcance:** controla duración y cancelación. **Planificador persistente:** conserva una intención de trabajo para que Android pueda ejecutarla. Resolver una solicitud en un hilo secundario no cubre automáticamente los otros aspectos.

Ejercicio: decidir cómo implementar una búsqueda mientras se escribe y una actualización nocturna de caché. La primera depende de la interacción actual. La segunda puede admitir demora, restricciones e intentos posteriores. Una periodicidad solicitada no debe prometer un horario exacto al usuario.

WorkManager: estados, condiciones y retry

Un **Worker** implementa el trabajo. Un **WorkRequest** describe la solicitud. WorkManager conserva y ejecuta esa solicitud cuando corresponde. Sus estados permiten informar al usuario sin confundir espera con fallo. [10]

Estado	Interpretación
ENQUEUED	Pendiente de ejecución o de un nuevo intento
RUNNING	Worker en ejecución
SUCCEEDED	Trabajo completado
FAILED	Trabajo terminado con fallo
CANCELLED	Trabajo cancelado
BLOCKED	Espera dependencias previas de una cadena

Condiciones y backoff

Las **constraints** expresan condiciones como conectividad o batería. CONNECTED indica conexión a una red, no disponibilidad de Internet ni respuesta del servidor. Si se pierde una condición, el sistema puede detener y reprogramar el trabajo. [11]

Backoff es la espera antes de reintentar. Puede ser lineal o exponencial. El intervalo mínimo de repetición para trabajo periódico es 15 minutos y su ejecución es inexacta. Para la demo se utiliza trabajo de una sola vez con backoff inicial de 30 segundos. [11]

Situación	Respuesta del ejemplo
IOException temporal	Retry con límite
HTTP 429	Evaluar retry y política del servicio
HTTP 500-599	Puede justificar retry limitado
HTTP 400 o 404	Fallo para esa solicitud
Contrato o JSON incompatible	Investigar y fallar sin bucle infinito
Cancelación	Propagarla y respetar el alcance

La decisión depende del servicio. Para 429, considerar Retry-After si está presente. Un retry genérico no debe bombardear la API. Mantener datos previos hasta terminar una actualización válida.

Nota: Data de WorkManager admite metadatos pequeños, con límite de tamaño. No usar inputData u outputData como base de datos ni como almacén de tokens. [12]

Implementación de un CoroutineWorker

El siguiente fragmento presenta una versión reducida del Worker del proyecto. Se supone que refreshPokemonList consulta Retrofit, escribe en Room y propaga errores. metadata guarda una fecha no sensible después del éxito.

```
class PokeSyncWorker(
    context: Context,
    params: WorkerParameters,
    private val refreshPokemonList: RefreshPokemonListUseCase,
    private val metadata: SyncMetadataStore
) : CoroutineWorker(context, params) {
    override suspend fun doWork(): Result = try {
        refreshPokemonList()
        metadata.markSuccess(System.currentTimeMillis())
        Result.success()
    } catch (cancelled: CancellationException) {
        throw cancelled
    } catch (_: IOException) {
        retryOrFail()
    } catch (error: HttpException) {
        if (error.code() == 429 || error.code() in 500..599)
            retryOrFail()
        else Result.failure()
    } catch (_: Exception) {
        Result.failure()
    }

    private fun retryOrFail(): Result =
        if (runAttemptCount < 3) Result.retry()
        else Result.failure()
}
```

Imports de plataforma: android.content.Context, androidx.work.CoroutineWorker y WorkerParameters. Errores: java.io.IOException, retrofit2.HttpException y kotlinx.coroutines.CancellationException. Los otros tipos pertenecen al proyecto.

Lectura del ejemplo

El contador comienza en cero. Con la política mostrada se permiten los intentos 0, 1, 2 y 3: hasta cuatro ejecuciones por fallos reintentables. El éxito se comunica después de persistir. La cancelación se vuelve a lanzar antes de capturar errores generales. [13]

En el proyecto completo se agregan progreso, códigos de error seguros y una simulación disponible solo en debug. No se exponen mensajes internos o payloads en la UI. Si Retrofit devuelve Response<T>, clasificar también respuestas HTTP no exitosas: no asumir que toda respuesta lanza HttpException.

Solicitudes únicas e idempotencia

El planificador de la demo construye una solicitud de una sola vez y espera conectividad. Las dependencias usan androidx.work:work-runtime-ktx:2.12.0 en el proyecto preparado. Revisar compatibilidad antes de reutilizarlo en otro proyecto. [14]

```
val request = OneTimeWorkRequestBuilder<PokeSyncWorker>()
    .setConstraints(
        Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .build()
    )
    .setBackoffCriteria(
        BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SECONDS
    )
    .build()

val wm = WorkManager.getInstance(context.applicationContext)
wm.enqueueUniqueWork(
    "poke-cache-sync", ExistingWorkPolicy.KEEP, request
)

// Acción independiente para cancelar.
wm.cancelUniqueWork("poke-cache-sync")
```

Imports: androidx.work.* y java.util.concurrent.TimeUnit. El constructor con dependencias adicionales del Worker requiere la fábrica de la sección siguiente.

Qué garantiza KEEP

KEEP conserva un trabajo con ese nombre mientras sigue pendiente o activo. Cuando termina, una solicitud posterior puede empezar. Esto limita duplicaciones de programación, pero no convierte la operación en una ejecución exactamente una vez. [10]

Idempotencia significa que repetir la operación no provoca efectos adicionales indeseados. En la demo, Room hace upsert por ID: volver a recibir el Pokémon 25 actualiza ese registro en vez de crear copias. Para pagos u otras escrituras remotas se necesitarían medidas adicionales del servidor.

La fecha de éxito y la caché se escriben en almacenes distintos. Si el proceso muere entre ambas escrituras, puede existir catálogo nuevo con fecha anterior. Una repetición segura permite recuperarse. Es una limitación explícita de la demo.

Integración con Clean Architecture y Koin

WorkManager puede crear un Worker después de que desaparezca la pantalla que lo solicitó. Sus dependencias deben poder reconstruirse desde el proceso y el almacenamiento. No pasar una Activity, un ViewModel o una lista en memoria como dependencia durable.

Distribución en el proyecto

Capa	Tipo del proyecto	Responsabilidad
Dominio	PokemonSyncScheduler	Contrato para solicitar y observar trabajo
Dominio	RefreshPokemonListUseCase	Actualizar la lista mediante el repositorio
Datos	PokeSyncWorker	Ejecutar y clasificar el resultado
Datos	PokeWorkerFactory	Construir nuevas instancias del Worker
Datos	WorkManagerPokemonSyncScheduler	Configurar y observar WorkManager
Presentación	PokeSyncViewModel	Exponer estado para Compose

Koin registra repositorio, caso de uso, fábrica y planificador. Una WorkerFactory recibe dependencias y crea una instancia nueva cuando WorkManager la solicita. Si no conoce la clase solicitada, puede devolver null para permitir el manejo correspondiente. [15]

```
// Fragmento de la Application con Configuration.Provider.
override val workManagerConfiguration: Configuration
    get() = Configuration.Builder()
        .setWorkerFactory(getKoin().get<PokeWorkerFactory>())
        .build()
```

En la configuración preparada se elimina solo el metadata de WorkManagerInitializer del InitializationProvider de AndroidX Startup. Koin inicia en Application.onCreate antes del primer acceso diferido a WorkManager. Evitar inicializarlo dos veces o quitar otros inicializadores sin razón. [15]

Para estudiar: la inyección facilita sustituir el repositorio por un falso durante pruebas. Persistencia e inyección resuelven problemas diferentes: reconstruir una dependencia no recupera por sí solo la información que guardaba en memoria.

Estado observable y demo de PokeAPI

La demo se abre desde **Demo: actualización en segundo plano** en la pantalla principal. La lista continúa leyendo Room. El Worker actualiza nombres e identificadores del catálogo; no descarga todas las imágenes para uso offline.

La pantalla muestra el estado, los intentos iniciados, la fecha del último éxito y una muestra de Pokémon guardados. La fecha se conserva en DataStore. El historial de WorkManager puede podarse y no debe tratarse como registro permanente de negocio.

```
// Fragmento directo para comprender la API.
// En el proyecto, el planificador adapta WorkInfo al dominio.
val flow = remember(wm) {
    wm.getWorkInfosForUniqueWorkFlow("poke-cache-sync")
}
val infos by flow.collectAsStateWithLifecycle(
    initialValue = emptyList()
)
val active = infos.firstOrNull { !it.state.isFinished }
Text(active?.state?.name ?: "Sin trabajo activo")
```

El fragmento requiere remember, getValue, Text, la extensión de lifecycle y un WorkManager recibido como wm. La ausencia de trabajo activo no distingue por sí sola éxito, fallo o cancelación. Para el resultado final, seleccionar explícitamente el registro correspondiente. [12]

Recorrido individual

1. Abrir la pantalla y anotar la cantidad de Pokémon local.
2. Solicitar actualización y comprobar el estado final y la fecha.
3. Sin otro trabajo activo, usar el botón de simulación en debug.
4. Observar que el primer intento falla y el trabajo espera retry.
5. Revisar cómo Room conserva el catálogo mientras espera.
6. Solicitar y cancelar una actualización pendiente.

La simulación falla una vez por solicitud. Si la consulta real posterior también falla, continúa la política limitada. Los botones de actualización anteriores de la app conservan su comportamiento de la sesión previa, ligado al ViewModel; la pantalla de demo muestra el camino persistente.

Pruebas y diagnóstico del trabajo

Una prueba del método doWork verifica lógica del Worker, pero no demuestra por sí sola que Android programe correctamente con red real, batería o process death. Combinar pruebas con repositorios falsos y verificaciones en un emulador. TestListenableWorkerBuilder permite preparar un CoroutineWorker con dependencias controladas. [16]

Prueba	Preparación	Resultado que se observa
Éxito	Repositorio falso que completa	Result.success
Fallo temporal	Repositorio que lanza IOException	Retry antes del límite
Límite	runAttemptCount igual a 3	Fallo final
HTTP permanente	Respuesta simulada 404	Fallo sin retry
Cancelación	Dependencia lanza CancellationException	Cancelación propagada

Pruebas de integración

Sin red, solicitar trabajo: debe quedar pendiente y conservar la caché. Restablecer conexión y observar ejecución cuando Android lo permita. Repetir pulsaciones mientras está pendiente para comprobar un único trabajo activo con el nombre acordado.

Para process death, llevar la app al background y usar el comando de prueba siguiente, sustituyendo el paquete si cambia:

```
adb shell am kill com.cuscatlan.myfirstbankingapp
```

Volver a abrir y comprobar reconstrucción. No confundir esta prueba con force-stop, rotación, desinstalación o borrado de datos. Las dos últimas eliminan información necesaria para recuperar el escenario.

Si el trabajo no empieza

Revisar conexión, restricciones, backoff, estado previo con KEEP, fábrica e inicialización. Background Task Inspector y las herramientas de diagnóstico de WorkManager ayudan a observar solicitudes. Evitar resolver una espera legítima con bucles de reintento en la UI. [17]

Evidencia de prueba: registrar escenario, versión de app, pasos, estado esperado y estado real. Una captura de ENQUEUED demuestra espera en ese momento, no un fallo permanente.

Almacenamiento y clasificación de datos

Elegir almacenamiento exige distinguir el valor y la sensibilidad del dato. Una caché pública puede tener necesidades distintas de un refresh token. Definir también retención, borrado, backup y recuperación.

Dato	Opción orientativa	Pregunta de revisión
Catálogo público	Room o archivo interno	¿Cuándo queda obsoleto?
Preferencia de vista	DataStore	¿Se conserva tras reiniciar?
Estado pequeño de pantalla	Estado guardado	¿Puede reconstruirse?
Credencial sensible	Minimizar y proteger persistencia	¿Qué riesgo crea su extracción?
Clave criptográfica	Android Keystore	¿Cómo se autoriza su uso?

Room facilita persistencia estructurada sobre SQLite con entidades, DAOs y migraciones. DataStore administra pequeños conjuntos de preferencias o datos tipados mediante APIs asíncronas. No deben asumirse equivalentes ni considerar que cifran automáticamente todo contenido. [18, 19]

Aislamiento y protección adicional

El almacenamiento interno participa del aislamiento de Android. Aun así, un dato puede filtrarse por copias de seguridad, logs, exportación de archivos o una función autorizada que muestra más de lo necesario. Para información sensible, el modelo de amenazas determina si hace falta cifrado adicional y cómo gestionar las claves. [20]

Modelo de amenazas es una descripción de qué se protege, quién podría atacarlo y por qué caminos. Para esta app, separar catálogo público de hipotéticas credenciales de una API privada.

Recuperación coherente

Un archivo cifrado restaurado sin su clave puede quedar inutilizable. Una sesión cerrada no debería reaparecer porque la UI conserva un token viejo. Especificar qué se elimina al cerrar sesión y qué puede mantenerse por ser público.

Ejercicio: documentar tres datos de la app con ubicación, sensibilidad, periodo de conservación y forma de borrado. No agregar cifrado indiscriminado sin poder explicar quién administra la clave y qué ocurre si se pierde.

Android Keystore y AES-GCM

Keystore administra claves criptográficas y permite restringir sus operaciones. La clave puede permanecer no exportable. El soporte de hardware y sus garantías dependen del dispositivo. El dato cifrado se guarda aparte, por ejemplo en un archivo interno. [21]

```
// Fragmento API 23+. Crear solo si el alias no existe.
val generator = KeyGenerator.getInstance(
    KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore"
)
generator.init(
    KeyGenParameterSpec.Builder("session-key",
        KeyProperties.PURPOSE_ENCRYPT or
            KeyProperties.PURPOSE_DECRYPT)
        .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
        .setEncryptionPaddings(
            KeyProperties.ENCRYPTION_PADDING_NONE)
        .build()
)
generator.generateKey()
```

Imports: javax.crypto.KeyGenerator y android.security.keystore.KeyGenParameterSpec, KeyProperties. Recuperar una clave existente antes de usarla; regenerarla en cada arranque impediría descifrar datos anteriores.

Cifrado autenticado

AES-GCM protege confidencialidad e integridad del contenido. Un **IV** es un valor por operación que no debe repetirse con la misma clave. Cipher puede generarlo al inicializar el cifrado. Guardar IV y ciphertext con su etiqueta, en un formato versionado. [20]

```
// key es una SecretKey recuperada de Keystore.
val cipher = Cipher.getInstance("AES/GCM/NoPadding")
cipher.init(Cipher.ENCRYPT_MODE, key)
val ciphertext = cipher.doFinal(plainBytes)
val iv = cipher.iv
```

Para descifrar se usa el mismo IV con GCMParameterSpec(128, iv). Los datos alterados deben producir error de autenticidad, no una recuperación silenciosa de texto. Si la clave queda invalidada, implementar recuperación, como borrar la sesión y autenticar nuevamente.

Límite: cifrar archivos no evita que una app comprometida use los datos mientras están disponibles en una operación legítima.

BiometricPrompt y autenticación local

BiometricPrompt ofrece un diálogo de autenticación del sistema. La app debe comprobar disponibilidad mediante canAuthenticate y manejar ausencia de enrolamiento, error y cancelación. La biometría local no concede automáticamente una sesión en el backend. [22]

```
val info = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Confirmar acceso")
    .setSubtitle("Usa tu biometría")
    .setAllowedAuthenticators(BIOMETRIC_STRONG)
    .setNegativeButtonText("Cancelar")
    .build()
prompt.authenticate(info)
```

El fragmento usa androidx.biometric y BIOMETRIC_STRONG de BiometricManager.Authenticators. prompt se construye previamente con FragmentActivity, executor y AuthenticationCallback. Declarar USE_BIOMETRIC y mantener el objeto con un host adecuado. Desde Compose, delegar la acción y evitar crear prompts en cada recomposición. [22]

Qué se protege realmente

Una pantalla puede decidir mostrarse tras un callback de éxito. Cuando la necesidad es proteger una operación criptográfica, vincular el uso de la clave con autenticación y un CryptoObject. Un booleano en memoria que dice autenticado no sustituye esa vinculación.

Si se admite credencial del dispositivo, revisar las combinaciones de autenticadores compatibles y los requisitos de las versiones de Android. No combinar opciones de botón negativo incompatibles con la credencial elegida.

Caso aplicado

Una app privada podría exigir autenticación local antes de descifrar una credencial persistida. Luego, al realizar una solicitud, el servidor verifica la sesión y los permisos del recurso. Son decisiones diferentes: presencia local del usuario, acceso a la clave y autorización remota.

En PokeAPI, el catálogo público no requiere biometría para actualizarse. Añadirla sin una necesidad concreta aumenta complejidad y dificulta la actualización automática cuando el usuario no está presente.

HTTPS, TLS y certificate pinning

HTTPS utiliza TLS para proteger comunicaciones. El cliente valida certificados y hostname para comprobar al servidor. Una configuración que acepta cualquier certificado o cualquier nombre elimina parte de esa protección. Mantener validaciones normales y evitar excepciones indiscriminadas. [23]

```
<!-- res/xml/network_security_config.xml -->
<network-security-config>
    <base-config cleartextTrafficPermitted="false" />
</network-security-config>
```

Dentro de la etiqueta application del manifest, añadir el atributo android:networkSecurityConfig="@xml/network_security_config". Comprobar que la pila de red empleada respeta esta configuración. La base URL de la demo es https://pokeapi.co/api/v2/.

Alcance de la protección

TLS protege el tránsito, pero no elimina datos sensibles de los logs ni cifra por sí mismo la base local. El servidor receptor sigue accediendo a los datos que recibe.

Pinning restringe las claves públicas aceptadas de una cadena de certificados. Puede reducir confianza en autoridades adicionales, pero introduce riesgo de interrupción si el servidor cambia claves y la app no está preparada. Requiere planificación, respaldo y recuperación. No se añade por defecto para la API pública ajena de la demo. [23]

Decisión	Consecuencia que debe evaluarse
Certificados de debug	Limitar excepciones a compilaciones de depuración
Rotación de clave del servidor	Verificar compatibilidad de clientes instalados
Expiración de pins	Evita cierto bloqueo, pero deja de aplicar pinning
Desactivar validación TLS	Expone la comunicación a ataques de intermediario

Para razonar: si una conexión falla por un certificado inválido, aceptar todos los certificados oculta el problema y debilita la aplicación. Corregir la causa y mantener una política de confianza deliberada.

Tokens, sesiones y autorización

Una sesión representa el contexto de acceso de un usuario o cliente. Un token es una credencial para acreditar determinadas condiciones de acceso. La app debe protegerlo y el servidor debe verificarlo y aplicar permisos sobre cada recurso. [24, 25]

Concepto	Responsabilidad
Autenticación	Comprobar identidad o credenciales
Autorización	Decidir qué operaciones y recursos se permiten
Access token	Acceso durante vigencia y alcance definidos
Refresh token	Obtener nuevos access tokens según protocolo
Revocación	Invalidar una credencial o sesión cuando corresponda

Renovación controlada

Ante un 401, una app con renovación puede intentar obtener una nueva credencial. Coordinar solicitudes simultáneas para evitar múltiples renovaciones y limitar la repetición de la llamada original. Si la renovación falla, llevar al usuario a recuperación de sesión. Un 403 no se resuelve necesariamente renovando: puede indicar falta de autorización.

No guardar contraseñas para repetir login silenciosamente. Si se utiliza OAuth para una app móvil, seguir el flujo recomendado por el proveedor para clientes públicos, como Authorization Code con PKCE. Una app distribuida no puede mantener un client secret confidencial en su binario. [25]

Errores frecuentes

- Confundir un JWT firmado con un contenido cifrado.
- Colocar tokens en URLs que pueden registrarse o compartirse.
- Limpiar solo la pantalla al cerrar sesión y conservar credenciales reutilizables.
- Autorizar acceso únicamente ocultando botones en la UI.

El servidor debe validar firma o mecanismo equivalente, vigencia y contexto del token según el protocolo. Logout limpia credenciales locales y solicita revocación cuando el servicio lo permite. Definir también vencimiento y recuperación de sesión. [24]

En la demo: PokeAPI no necesita tokens. Este apartado complementa el estudio para aplicaciones con APIs privadas; no implica que deban inventarse credenciales para consultar el catálogo público.

R8, logs y secretos

R8 elimina código sin uso, optimiza y puede ofuscar nombres. Probar una compilación release es necesario porque la reflexión y la serialización pueden requerir reglas específicas. Conservar mapping.txt para interpretar fallos ofuscados. [26]

```
// build.gradle.kts de app. DSL para AGP 9.3 o superior.
android {
    buildTypes {
        release {
            optimization { enable = true }
        }
    }
}
// Reglas propias: src/main/keepRules/custom-rules.keep
```

Para AGP anterior a 9.3 existe la DSL isMinifyEnabled/isShrinkResources y proguardFiles. La versión del proyecto determina qué configuración aplicar. No usar keep de todo el código como respuesta automática a cualquier fallo. [26]

Logs que ayudan sin exponer

Un evento como actualización terminada y un código de error pueden ser suficientes para diagnóstico. Revisar el contenido de los interceptores HTTP, las cabeceras Authorization, cookies, payloads y reportes de fallos. Evitar logging BODY en release y no registrar datos sensibles. Un nivel BASIC también puede incluir URLs: revisar sus parámetros. [27]

Secretos distribuidos

BuildConfig, strings.xml, variables de CI y archivos fuera de Git no garantizan confidencialidad si sus valores terminan dentro del APK. La ofuscación dificulta lectura casual, pero un secreto del servidor debe permanecer en el backend. Para claves públicas de servicio, aplicar restricciones y cuotas cuando corresponda. [28]

Caso: el repositorio deja de contener una clave, pero el build la inserta como constante. Se redujo exposición en Git, no la posibilidad de extraerla de la app. Revisar por separado el repositorio, la configuración del build y el binario distribuido.

Screenshots y clipboard

La información sensible también puede salir mediante funciones ordinarias de la interfaz. Antes de aplicar controles, identificar qué pantalla y qué dato necesitan protección. Bloquear capturas del catálogo público puede perjudicar el uso sin aportar una defensa relevante.

Capturas de pantalla

FLAG_SECURE permite limitar capturas y presentación en pantallas no seguras para la ventana protegida. En una app con varios destinos dentro de una Activity, administrar el flag según la pantalla sensible y restaurar la política al salir. No protege frente a una cámara externa ni elimina todos los riesgos de dispositivos comprometidos. [29]

```
// En la Activity que aloja la pantalla sensible.
window.addFlags(WindowManager.LayoutParams.FLAG_SECURE)
// Cuando la política de la app permita retirar la protección:
window.clearFlags(WindowManager.LayoutParams.FLAG_SECURE)
```

Portapapeles

El clipboard facilita transferir contenido fuera de la app. Evitar copiar tokens o contraseñas automáticamente. En versiones compatibles, una marca de sensibilidad permite ocultar la vista previa del contenido, pero no cifra ni revoca lo copiado. [30]

```
// API 33+. clip es ClipData y clipboard es ClipboardManager.
clip.description.extras = PersistableBundle().apply {
    putBoolean(ClipDescription.EXTRA_IS_SENSITIVE, true)
}
clipboard.setPrimaryClip(clip)
```

Imports: android.view.WindowManager, android.os.PersistableBundle y android.content.ClipDescription. Si se soportan versiones anteriores, proteger el acceso a constantes según API y revisar el mecanismo compatible indicado por Android.

Ejercicio: explicar por qué ocultar una vista previa es distinto de impedir lectura del contenido. Después decidir qué haría una app al ofrecer copiar un código temporal y cómo evitaría incluir otros datos de la sesión.

OWASP MASVS y revisión por riesgos

MASVS es un estándar de verificación de seguridad móvil. Describe controles organizados por áreas. MASTG complementa el estudio con guía y pruebas, y MASWE clasifica debilidades. Una revisión breve de clase cubre una selección y no certifica la aplicación. [31]

Grupo	Área de estudio	Ejemplo de evidencia en el proyecto
STORAGE	Datos en el dispositivo	Inventario de caché y política de borrado
CRYPTO	Uso de criptografía y claves	Algoritmo, IV y recuperación de clave
AUTH	Autenticación y autorización	Flujo de sesión cuando exista
NETWORK	Comunicación	HTTPS y validación de confianza
PLATFORM	Interacción con Android	Permisos, componentes y enlaces
CODE	Calidad y mantenimiento	Dependencias y pruebas de release
RESILIENCE	Ingeniería inversa y manipulación	Evaluación de protecciones pertinentes
PRIVACY	Tratamiento de datos personales	Minimización y finalidades justificadas

Cómo registrar un hallazgo

Usar cuatro elementos: **riesgo**, **evidencia**, **corrección** y **prueba posterior**. Ejemplo hipotético: el interceptor de red expone una credencial en un log de release; se redacta la cabecera en el log y se elimina el registro sensible; se repite una solicitud de prueba y se comprueba su ausencia.

Una vulnerabilidad requiere evidencia contextual. Si una app solo muestra Pokémon públicos, no se debe concluir que necesita autenticación biométrica para cada lectura. La pertinencia depende del dato, las operaciones y las amenazas.

Actividad de análisis: elegir dos grupos y proponer una prueba para cada uno. La frase usa HTTPS es una configuración declarada. La evidencia puede incluir configuración inspeccionada y resultados de una prueba de tráfico en un entorno autorizado.

Checklist y práctica de consolidación

Completar cada punto con **cumple**, **pendiente** o **no aplica**, incluyendo justificación y evidencia. El checklist es una adaptación para aprendizaje, no una transcripción completa de MASVS. [31]

Área	Pregunta de revisión
Datos	¿Se identifican datos públicos, personales y credenciales?
Persistencia	¿La caché se conserva al fallar la red? ¿Hay política de borrado?
Backups	¿Se revisó qué se respalda y cómo se recupera?
Red	¿Se mantiene TLS y se bloquean excepciones inseguras?
Sesión	¿El backend autoriza recursos? ¿Logout elimina credenciales locales?
Plataforma	¿Los permisos y componentes exportados son necesarios?
Entradas	¿Los enlaces rechazan rutas y parámetros inesperados?
Exposición	¿Logs, capturas y clipboard reciben una política según riesgo?
Release	¿Se probó el build distribuible y se revisaron secretos?
Background	¿Hay unicidad, idempotencia, cancelación y retry limitado?

Práctica con la app del curso

Mantener Room como fuente de la lista. Solicitar una actualización única con conectividad y backoff. Mostrar espera, ejecución y resultado. Probar la caché sin conexión y reconstruir observación después de recrear la app.

Registrar dos mejoras de seguridad pertinentes. Por ejemplo, bloquear HTTP claro y revisar el logging de release. Si una medida no aplica, justificarlo: el catálogo público no implica por sí mismo una sesión de usuario.

Criterios de aceptación

- Pulsaciones repetidas no crean varios trabajos activos del mismo nombre.
- Sin red, el trabajo espera y los datos anteriores continúan disponibles.
- Un error temporal reintenta hasta el límite definido y un error permanente termina.
- La cancelación no anuncia éxito y la UI recupera estado al volver.
- La entrega incluye configuración, pruebas y límites documentados.

Entregables sugeridos: código, README con pasos reproducibles y evidencias del checklist. Explicar qué está implementado y qué sigue pendiente evita confundir una demo funcional con una revisión integral de seguridad.

Autoevaluación

Responder sin consultar el material. Luego comparar con las respuestas comentadas de la página siguiente.

1. ¿Por qué guardar una operación importante solo en onDestroy puede perder información?

2. Una búsqueda debe restaurarse y una lista debe seguir disponible offline. ¿Qué mecanismos elegirías para cada dato?

3. ¿Qué diferencia existe entre una recomposición, una recreación de Activity y process death?

4. ¿Qué permiso necesita la consulta de PokeAPI? ¿Hay que mostrar un diálogo runtime?

5. ¿Qué añade un App Link verificado y qué controles siguen siendo responsabilidad de la app y del servidor?

6. ¿Por qué una coroutine de ViewModel puede no cubrir una actualización que debe continuar al abandonar la pantalla?

7. Un Worker queda en ENQUEUED con CONNECTED. Menciona dos causas posibles sin concluir que está roto.

8. Con runAttemptCount menor que 3 para devolver retry, ¿cuántos intentos totales permite la política del ejemplo?

9. ¿Por qué KEEP e idempotencia resuelven problemas relacionados, pero distintos?

10. ¿Qué almacena Keystore? ¿Qué se guarda junto al ciphertext cuando se usa AES-GCM?

11. ¿Por qué el éxito de BiometricPrompt no demuestra autorización para una operación privada del backend?

12. ¿Qué riesgo introduce aceptar todos los certificados para arreglar un error de conexión?

13. Una clave se retira de Git y se inserta con BuildConfig. ¿Qué exposición se corrigió y cuál permanece?

14. ¿Qué diferencias hay entre FLAG_SECURE y marcar el clipboard como sensible?

15. Proponer un hallazgo de seguridad con riesgo, evidencia, corrección y prueba posterior.

Caso integrador

Una app inicia la descarga dentro del composable, guarda tokens en logs, borra la caché antes de descargar y repite cualquier error para siempre. Identificar cuatro problemas, proponer correcciones y explicar cómo verificar cada una. No asumir que cambiar una biblioteca resuelve toda la arquitectura.

Respuestas comentadas

1. **onDestroy puede no ejecutarse** cuando termina el proceso. Persistir la operación o su resultado cuando corresponde, antes de depender del cierre.

2. Consulta pequeña con estado restaurable, por ejemplo SavedStateHandle. Lista en Room. La elección también depende de quién es propietario del estado y de su duración.

3. Recomposición actualiza UI; recreación sustituye una instancia de Activity; process death elimina objetos del proceso, incluidos ViewModels.

4. INTERNET en manifest. No necesita diálogo runtime para esa consulta.

5. Añade asociación verificada entre dominio y app. Siguen siendo necesarias validación de entradas y autorización sobre recursos privados.

6. Su alcance puede cancelarse al eliminar el ViewModel y no persiste por sí solo una solicitud tras process death. Evaluar WorkManager para trabajo durable diferible.

7. Puede esperar conexión o backoff. También influyen las políticas de Android. CONNECTED no demuestra disponibilidad del servidor.

8. **Hasta cuatro:** el contador comienza en cero y los intentos son 0, 1, 2 y 3.

9. KEEP evita programaciones simultáneas del mismo nombre mientras existe trabajo pendiente. Idempotencia permite repetir la operación sin efectos indeseados. No hay garantía general de ejecución exactamente una vez.

10. Keystore administra claves. Se conserva IV y ciphertext con su etiqueta de autenticación en un formato apropiado. Nunca repetir IV con la misma clave.

11. La biometría es local. El servidor aún debe validar sesión y permisos del recurso.

12. Se debilita la comprobación de identidad del servidor y se facilita un ataque de intermediario.

13. Mejora la exposición en el repositorio. El valor distribuido dentro del APK sigue pudiendo extraerse.

14. FLAG_SECURE protege la ventana frente a ciertas capturas o pantallas. La marca del clipboard oculta la vista previa compatible y no cifra su contenido.

15. Una respuesta válida incluye evidencia reproducible. Ejemplo: credencial registrada por un interceptor; retirar ese registro y comprobar su ausencia con datos ficticios.

Caso integrador: mover la coordinación fuera del cuerpo del composable; eliminar logs sensibles; actualizar la caché sin borrar datos útiles antes del éxito; clasificar errores y limitar retry. Probar recomposición, logs de release, fallo de red y agotamiento de intentos.

Glosario

Término	Definición de estudio
Activity	Componente que aloja una experiencia de interfaz
Lifecycle	Estados y transiciones de un componente
Process death	Terminación del proceso y pérdida de objetos en memoria
Recomposición	Nueva ejecución de composables para actualizar UI
Coroutine	Unidad de concurrencia que puede suspenderse
Scope o alcance	Contexto que controla duración y cancelación
Persistencia	Conservación de información más allá de objetos en memoria
Caché	Copia local destinada a facilitar acceso a datos
Intent	Mensaje que expresa una acción entre componentes
Deep link	Enlace a un destino específico de una app
App Link	Enlace web con asociación verificada de dominio y app
Worker	Implementación de trabajo ejecutable por WorkManager
Constraint	Condición que debe cumplirse para ejecutar trabajo
Retry	Solicitud de un nuevo intento tras un fallo
Backoff	Espera entre intentos
Idempotencia	Repetición sin efectos adicionales indeseados
Keystore	Sistema para administrar y usar claves criptográficas
IV	Valor por operación requerido por ciertos modos de cifrado
Ciphertext	Contenido resultante del cifrado
TLS	Protocolo para proteger comunicaciones en tránsito
Pinning	Restricción adicional de claves o certificados aceptados
Token	Credencial de acceso con condiciones de validación
Ofuscación	Transformación que dificulta la lectura del código
MASVS	Estándar de verificación de seguridad móvil de OWASP

Conexión entre conceptos: la UI observa una caché persistente; WorkManager programa su actualización; el Worker usa un repositorio; TLS protege la consulta; el checklist revisa entradas y posibles exposiciones. Cada mecanismo cubre una parte y debe evaluarse junto a los demás.

Fuentes oficiales: plataforma y trabajo

Referencias consultadas el 1 de noviembre de 2026. Las versiones y requisitos pueden cambiar. Cada título abre la documentación correspondiente.

[1] The activity lifecycle
Android Developers

[2] Save UI state in Compose
Android Developers

[3] State and Jetpack Compose
Android Developers

[4] Request runtime permissions
Android Developers

[5] ViewModel overview
Android Developers

[6] Intents and intent filters
Android Developers

[7] About deep links
Android Developers

[8] Verify App Links
Android Developers

[9] Task scheduling with WorkManager
Android Developers

[10] Managing work
Android Developers

[11] Define work requests
Android Developers

[12] WorkManager API reference
Android Developers

[13] Threading in CoroutineWorker
Android Developers

[14] WorkManager releases
Android Developers

[15] Custom WorkManager configuration
Android Developers

[16] Testing Worker implementation
Android Developers

[17] Debug WorkManager
Android Developers

Fuentes oficiales: datos y seguridad

Referencias consultadas el 1 de noviembre de 2026. Las versiones y requisitos pueden cambiar. Cada título abre la documentación correspondiente.

[18] Save data using Room
Android Developers

[19] DataStore
Android Developers

[20] Cryptography
Android Developers

[21] Android Keystore system
Android Developers

[22] Show a biometric authentication dialog
Android Developers

[23] Network security configuration
Android Developers

[24] Session Management Cheat Sheet
OWASP

[25] OAuth 2.0 for Native Apps, RFC 8252
IETF / RFC Editor

[26] Enable app optimization with R8
Android Developers

[27] Log Info Disclosure
Android Developers

[28] Hardcoded Cryptographic Secrets
Android Developers

[29] Secure sensitive activities
Android Developers

[30] Secure Clipboard Handling
Android Developers

[31] OWASP Mobile Application Security Verification Standard
OWASP